

EVENTSPHERE PORTAL

Presentation Q&A Guide

An exhaustive compilation of 40 Quality Assurance and Technical System review questions and answers.

Author: EventSphere Core Team

Scope: Attendee POV, Technical Architecture, & Testing Layers

Status: Verified QA Complete

Part 1: User Point of View (User POV)

Q: 1. What is the core problem statement that EventSphere solves?

A: Traditional ticketing websites are cluttered, generic, and charge high service fees. They lack premium styling and don't provide a focused, exclusive booking environment. EventSphere solves this by providing a clean, aesthetic, and distraction-free portal designed specifically for premium, curated, or invite-only events.

Q: 2. Who is the target audience for this platform?

A: Organizers: Luxury brands, art galleries, fashion houses, wellness centers, and corporate event planners looking for a premium listing portal.

Attendees: Discerning guests who value high-end curations and want a smooth, aesthetic booking experience.

Q: 3. How does this website make event discovery easier for a user?

A: It groups events into high-end 'Curation Categories' (like *Galas & Soirées*, *Art & Exhibitions*, *Haute Couture*). Users can filter events instantly by typing a keyword, city, or date, avoiding overwhelming lists of irrelevant events.

Q: 4. What are the key features currently working on the platform?

A: Account creation, secure login, and email OTP dispatch simulation; searching and filtering events by title, tag, category, and city; creating events with banner and gallery uploads (with local disk fallback); booking tickets and generating unique, secure invitation passcodes; and attendee dashboard showing active entry passes, wishlist items, and account settings.

Q: 5. What features are currently under construction or simulated?

A: Real Payment Gateway: Ticket checkout is currently simulated using a complimentary pass-generation checkout flow.

Camera QR Scanner: The check-in validation currently relies on typing the passcode text rather than scanning via a mobile camera.

Q: 6. When and how will you integrate a payment gateway?

A: We plan to integrate **Stripe API** or **Razorpay SDK** in the next phase. The backend will generate a secure payment token, redirect the user to a secure payment page, and use a webhook to automatically issue the ticket pass code once payment succeeds.

Q: 7. How does an organizer manage bookings and guest check-ins?

A: Organizers have a dedicated dashboard showing a list of their created events. By clicking on an event, they can view the guest list, verify individual invitation codes, and toggle the guest's check-in status.

Q: 8. What is the 'Wishlist' feature, and how does it benefit the user?

A: The Wishlist lets users bookmark events they are interested in without committing to booking a ticket. It allows them to curate their upcoming experiences and easily purchase passes later.

Q: 9. How does this site make things easier for mobile users?

A: The layout is built using mobile-first CSS grid systems. Pages, forms, navigation bars, and buttons adapt dynamically to fit mobile viewports, allowing users to browse events and present check-in codes at doors on their phones.

Q: 10. Can an attendee cancel or refund a booking?

A: Currently, attendees can view their active passes in their dashboard. Toggling cancellations or initiating automatic refunds is planned for the payment integration phase.

Q: 11. How are invitation codes shared with attendees?

A: Upon successful checkout, a unique alphanumeric invitation passcode is generated and displayed on their dashboard. It is also simulated as being dispatched to their registered email for convenience.

Q: 12. What happens if an event sells out?

A: The system tracks the capacity limit (capacity) of each event. When the number of booked tickets reaches the capacity limit, the frontend replaces the 'Book Ticket' button with a 'Sold Out' badge, preventing overselling.

Q: 13. How does this platform differ from standard sites like Eventbrite?

A: EventSphere focuses on high-end design aesthetics (using a luxury cream, gold, and dark charcoal theme), offers a cleaner interface, and includes customized organizer features like private invitation code tracking.

Q: 14. What value does EventSphere bring to a luxury brand organizer?

A: It protects the brand's premium image. Listing an elite fashion show or art exhibit on a cluttered ticketing site dilutes its prestige; EventSphere provides a curated environment that aligns with luxury branding.

Q: 15. What are your monetization models? How can this project generate revenue?

A: Commission per Ticket: Taking a small fee (e.g., 2-3%) on paid tickets.

Featured Event Placements: Charging organizers to feature their event banner on the homepage.

Custom Branding Add-ons: Charging for custom email invites or custom RSVP forms.

Q: 16. What is the purpose of the 'Curation Categories'?

A: To maintain a premium theme. Instead of standard categories like 'seminars' or 'business workshops', EventSphere uses curated themes like *Haute Couture*, *Wellness Retreats*, and *Classical Concerts*.

Q: 17. How secure is a user's personal information on this site?

A: Passwords are encrypted before database storage, communication is encrypted using HTTPS protocols, and JWT tokens are used to authorize account access.

Q: 18. Can an organizer set up a free or complimentary event?

A: Yes. If the organizer sets the ticket price to 0, the checkout system automatically bypasses any payment simulations and issues a complimentary pass instantly.

Q: 19. How does the site handle different user roles (Attendee vs. Organizer)?

A: On signup, users select their role. Attendees get a dashboard focused on tickets and wishlists. Organizers get a dashboard for listing events, uploading media, and tracking check-ins.

Q: 20. What are the planned future enhancements for this project?

A: Implementing real Stripe payment checkouts; adding a camera-based QR code reader for door entry check-ins; integrating calendar exports (Google Calendar, iCal) for booked passes; and adding real-time email triggers via Brevo/Sendinblue.

Part 2: Technical & Teacher Point of View

Q: 1. What is the full technology stack of this application?

A: We use the standard **MERN** stack:

- **MongoDB:** NoSQL database.
- **Express.js:** Minimalist web server framework.
- **React.js** (built with Vite): Component-based frontend.
- **Node.js:** JavaScript runtime environment.

Q: 2. Why did you choose Vite over Create React App (CRA)?

A: Vite utilizes native ES modules, leading to near-instantaneous startup times and extremely fast Hot Module Replacement (HMR) during development. Production bundling is optimized using Rolldown/Esbuild, yielding smaller asset chunks than CRA's Webpack configuration.

Q: 3. Why MongoDB instead of a relational database like MySQL or PostgreSQL?

A: Dynamic Schema: Events contain variable attributes (e.g., coordinates, optional gallery image arrays, varying tag structures). Storing this in MongoDB avoids rigid tables and complex joins.

Developer Velocity: MongoDB stores data as JSON-like documents, which map directly to JavaScript objects, eliminating the need for object-relational mapping (ORM) translations.

Q: 4. Explain your database schema design and relationships.

A: We use **Mongoose** to define schemas:

- **User:** Holds login credentials, role, and bio details.
- **Event:** Refers to organizer (User ObjectId reference) and holds details like category, images, capacity, and city.
- **Ticket:** Links a user reference to an event reference and holds the unique inviteCode passcode.
- **Category:** Stores lookup items for categorizing events.

Q: 5. How do you handle password encryption?

A: We use **bcryptjs**. During registration (and profile password updates), the plain-text password is mixed with a random string (salt) with 10 rounds and hashed. The plain-text password is never saved or logged.

Q: 6. Explain your authentication flow.

A: We use **JSON Web Tokens (JWT)**:

1. The user logs in, and the server generates a token containing their user ID signed with a server-side secret.
2. The frontend stores this token in LocalStorage and attaches it as a Bearer token in the Authorization header of subsequent API requests.
3. A backend protect middleware decodes the token to verify the user's identity.

Q: 7. How does the role authorization middleware work?

A: We built an `authorize(...roles)` middleware that checks the authenticated user's role: if their role is not included in the allowed list, it returns a 403 Forbidden status code, protecting admin/organizer routes from unauthorized access.

Q: 8. How did you design the media upload storage architecture?

A: We created a hybrid system using **Multer** and **Cloudinary**:

1. Multer parses multipart form data and saves the file temporarily in a local Backend/uploads folder.
2. If Cloudinary credentials exist in `.env`, the script uploads the local file to Cloudinary and deletes the temp file.
3. If credentials are missing, it falls back to serving the file locally via Express static routing.

Q: 9. What is the 'Automatic Fallback Mode' in the server, and why did you build it?

A: In local development, database connection issues (like IP whitelisting errors on MongoDB Atlas) can crash the server. We built an in-memory database fallback using a mock database class. If the Atlas connection fails, the server automatically starts in mock mode, saving data in memory so testing is uninterrupted.

Q: 10. How is client-side state managed in React?

A: We use **Redux Toolkit** (slices for auth and events). Redux manages global states like current login status, user object, and fetched event lists, which prevents prop-drilling across layouts.

Q: 11. How do you handle API calls between the frontend and backend?

A: We use an **Axios instance** (`Frontend/src/services/api.js`). It is configured with a base URL matching our backend server and includes an interceptor that automatically attaches the JWT token to the Authorization header if it exists.

Q: 12. How does the search and filter query work at the database level?

A: The backend extracts query parameters (search, category, city, priceMax) from the URL. It builds a dynamic Mongoose query object using regex match: title: { \$regex: search, \$options: 'i' } and comparison operators: price: { \$lte: priceMax }.

Q: 13. How did you handle Cross-Origin Resource Sharing (CORS)?

A: We integrated the cors middleware in Express. It is configured to allow requests from specific origins (like the local frontend port or the deployed Render static site) and permits headers like Authorization and Content-Type.

Q: 14. What are React Router 'Protected Routes'?

A: It is a wrapper component (ProtectedRoute.jsx) that checks if the user is authenticated in the Redux store. If the user is logged in, it renders the dashboard page. If not, it redirects them to /login using the React Router <Navigate> component.

Q: 15. How do you prevent database queries from blocking the Express event loop?

A: All database queries are asynchronous. We use modern async/await syntax with Mongoose operations, which offloads the query operations to Node's background thread pool, allowing Express to continue processing incoming HTTP requests.

Q: 16. What is the purpose of `package-lock.json`?

A: It locks the exact versions of all nested dependencies installed in the node_modules folder. This ensures that the application builds identically across all developers' machines and production servers, preventing breaking changes from auto-updated packages.

Q: 17. How do you validate form input values on the server?

A: We use HTML5 native validations on the frontend, and Mongoose schema-level validations (e.g. required: true properties and email format validation match expressions) on the backend to prevent corrupted records.

Q: 18. Explain how static files are served in the application.

A: In the backend, we use Express's built-in static middleware: app.use('/uploads', express.static(path.join(__dirname, 'uploads'))). This tells Express that any HTTP requests targeting `/uploads` should be mapped directly to the files stored in the local server directory.

Q: 19. How did you resolve the build errors when deploying to Render?

A: We resolved build errors by ensuring that: we ran production builds locally using Vite to catch syntax and import errors; all dependencies were declared in package.json; and no absolute system filepaths were referenced in the codebase, using relative path resolutions instead.

Q: 20. How would you optimize the performance of EventSphere in a production environment?

A: Code Splitting: Using React lazy() and Suspense to load dashboard routes only when visited.

Image Compression: Resizing user uploads on the client side before sending them to the server.

Database Indexing: Adding indexes to fields like city and category in MongoDB to speed up query filtering.